Chapter 4

MPI Programming Patterns

4.1 Learning Objectives

After this chapter, students should be able to:

- select a collective operation by where its input begins and where its result is needed;
- distinguish Python-object collectives from typed NumPy-buffer collectives;
- distribute uneven data with counts and displacements;
- complete nonblocking requests without accessing active communication buffers;
- implement a one-dimensional halo exchange with correct neighbors and tags;
- estimate latency, bandwidth, synchronization, and load-balance costs; and
- validate an MPI program with one rank, uneven work, and empty partitions.

4.2 From Point-to-Point to Group Communication

Chapter 3 built communication protocols with explicit sends and receives. Many programs repeatedly broadcast an input, distribute work, and combine results. MPI provides *collective operations* for these patterns. Every process in the relevant communicator must participate in the operation, with compatible arguments and a compatible call order. A collective is not a substitute for agreeing on the data layout or the meaning of the result.

Operation	Input and result	Typical use
`bcast`	One root to all ranks	Share configuration or parameters.
`scatter`	One root sends one piece per rank	Distribute Python-object chunks.
`gather`	One piece per rank to one root	Collect local outputs for inspection.
`reduce`	Combine local values at one root	Produce a final sum or maximum.
`allreduce`	Combine local values at every rank	Give every rank a global convergence value.
`barrier`	Every rank waits for arrivals	Delimit a coordinated phase when needed.

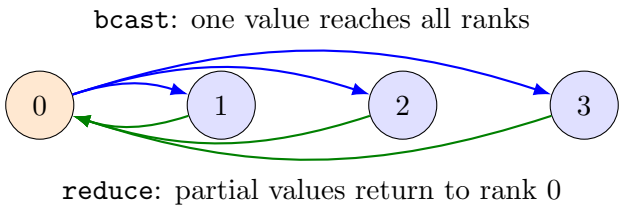


Figure 4.1: The direction of a broadcast differs from that of a reduction. These arrows show logical data flow, not MPI's internal network algorithm.

4.2.1 Root, Result, and Participation

The `root` argument names a rank *within the communicator*. In `comm.bcast(value, root=0)`, rank 0 supplies the useful value and every rank receives it. In `comm.reduce(local, op=MPI.SUM, root=0)`, every rank supplies a local value; only rank 0 receives the combined result. A nonroot `reduce` return value is not the global result. Use `allreduce` if every rank needs that result.

All ranks must call compatible collectives in the same sequence on the same communicator. If rank 0 calls `bcast` while rank 1 calls `reduce`, the program can hang or fail. A collective does not necessarily act as a general-purpose barrier: its completion guarantees the operation's own local completion semantics, not that every process has left the call. Add a barrier only when the algorithm or a carefully designed measurement actually needs one.

4.2.2 Objects and Numerical Buffers

mpi4py uses lower-case methods for general Python objects and upper-case methods for typed buffers. Python-object methods serialize their values. Upper-case methods communicate contiguous numerical storage directly and require compatible datatypes and capacities.

Feature	Objects	Typed buffers
Example	`comm.scatter(chunks)`	`comm.Scatterv(...)`
Input	Lists, dictionaries, scalars	NumPy arrays or other buffer objects
Receive storage	Returned Python object	Preallocated, compatible array
Uneven chunks	Build a list of unequal objects	Supply counts and displacements

Choose objects for small control data and clear introductory examples. Use buffer operations when large numerical arrays make serialization and copies important. Do not pair an object send with a buffer receive.

4.2.3 Where Gather and Reduction Results Appear

`gather` collects one Python object from every rank and returns a list only at the root. `Gatherv` writes unequal typed blocks into a preallocated root array using the same counts and element displacements as `Scatterv`. By contrast, `reduce` combines local values into one root result; `allreduce` gives that combined value to every rank. All four calls require participation from every rank in the communicator.

Operation	Result location	What the result contains
`gather`	Root only	A list of one object from each rank.
`Gatherv`	Root only	Unequal typed blocks in one receive array.
`reduce`	Root only	One combined value.
`allreduce`	Every rank	The same combined value on every rank.

The runnable example below constructs uneven blocks of the numbers $1, \ldots, N$. It uses an empty rank's $-\infty$ as the neutral contribution to a maximum, so $N < P$ remains valid. Notice that only rank 0 allocates the full receive array for `Gatherv`; every rank still calls the collective.

Listing 4.1: `collective_results.py`: gather, Gatherv, reduce, and allreduce

```python
"""Compare root-only and all-rank collective results on uneven blocks."""

import sys

import numpy as np
from mpi4py import MPI


comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if rank == 0:
    try:
        if len(sys.argv) > 2:
            raise ValueError
        n = int(sys.argv[1]) if len(sys.argv) == 2 else 7
        if n < 1:
            raise ValueError
    except ValueError:
        n = None
else:
    n = None

n = comm.bcast(n, root=0)
if n is None:
    if rank == 0:
        print("Usage: collective_results.py [N], where N >= 1")
    raise SystemExit(1)

q, remainder = divmod(n, size)
counts = np.array([q + (r < remainder) for r in range(size)],
                  dtype=np.int32)
displacements = np.zeros(size, dtype=np.int32)
displacements[1:] = np.cumsum(counts[:-1])

start = int(displacements[rank])
count = int(counts[rank])
local = np.arange(start + 1, start + count + 1, dtype=np.float64)

# Object gather returns a list on the root; other ranks receive None.
local_counts = comm.gather(count, root=0)
```

```
43
44 # Typed Gatherv writes the numerical blocks into preallocated root storage.
45 reconstructed = np.empty(n, dtype=np.float64) if rank == 0 else None
46 comm.Gatherv(local, [reconstructed, counts, displacements, MPI.DOUBLE]
47             if rank == 0 else None, root=0)
48
49 # A reduction has a result only at the root.
50 total = comm.reduce(float(local.sum()), op=MPI.SUM, root=0)
51
52 # An all-reduction makes the same maximum available on every rank.
53 local_max = float(local.max()) if count else float("-inf")
54 global_max = comm.allreduce(local_max, op=MPI.MAX)
55
56 if rank == 0:
57     expected = np.arange(1, n + 1, dtype=np.float64)
58     passed = (local_counts == counts.tolist()
59               and np.array_equal(reconstructed, expected)
60               and total == float(expected.sum())
61               and global_max == float(n))
62     print(f"N={n} P={size} counts={local_counts} "
63           f"sum={total:.0f} max={global_max:.0f} "
64           f"{'PASS' if passed else 'FAIL'}")
```

Run `mpiexec -n 4 python -m mpi4py collective_results.py 7`. The root should report counts `[2, 2, 2, 1]`, sum 28, maximum 7, and `PASS`. With $N = 3, P = 4$, rank 3 has an empty block but still participates in all four collectives. In an iterative solver, `allreduce` is appropriate for a global stopping condition because every rank must make the same decision.

4.3 Worked Program: Parallel Average

4.3.1 Why Reduce Both Sum and Count?

The average of rank averages is generally wrong when ranks own different numbers of elements. For example, one rank might own five numbers and another four. The correct global average is

$$\bar{x} = \frac{\sum_{r=0}^{P-1} s_r}{\sum_{r=0}^{P-1} c_r},$$

where s_r is rank r's local sum and c_r is its local count. Reducing both quantities also handles an empty partition: its sum and count are both zero.

The following program uses `bcast` for N, `scatter` for uneven Python lists, and two reductions. Rank 0 builds the data $1, 2, \ldots, N$ so that the expected average is easy to check.

Listing 4.2: `collective_average.py`: broadcast, scatter, and reduction

```
1  import sys
2  from mpi4py import MPI
3
4
5  comm = MPI.COMM_WORLD
6  rank = comm.Get_rank()
7  size = comm.Get_size()
8
9  if rank == 0:
10     try:
11         if len(sys.argv) > 2:
12             raise ValueError
```

```
        n = int(sys.argv[1]) if len(sys.argv) == 2 else 17
        if n < 1:
            raise ValueError
    except ValueError:
        n = None
else:
    n = None

n = comm.bcast(n, root=0)
if n is None:
    if rank == 0:
        print("Usage: collective_average.py [N], where N >= 1")
    raise SystemExit(1)

if rank == 0:
    data = list(range(1, n + 1))
    q, remainder = divmod(n, size)
    counts = [q + (r < remainder) for r in range(size)]
    starts = [r * q + min(r, remainder) for r in range(size)]
    chunks = [data[s:s + c] for s, c in zip(starts, counts)]
else:
    data = None
    chunks = None

local = comm.scatter(chunks, root=0)
local_sum = sum(local)
local_count = len(local)

global_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
global_count = comm.reduce(local_count, op=MPI.SUM, root=0)

if rank == 0:
    average = global_sum / global_count
    expected = sum(data) / len(data)
    passed = global_sum == sum(data) and average == expected
    print(f"N={n} P={size} sum={global_sum} "
          f"average={average:.2f} {'PASS' if passed else 'FAIL'}")
```

From `examples/chapter4`, run:

```
mpiexec -n 4 python -m mpi4py collective_average.py 17
```

The expected output includes `sum=153`, `average=9.00`, and `PASS`. With $N = 3$ and four ranks, one local list is empty but contributes zero to both reductions. With one rank, the same collective calls remain valid. This example is intentionally small; constructing all data and chunks at rank 0 costs memory and makes rank 0 a distribution bottleneck.

Rank	Owned values	Count c_r	Sum s_r	Local average
0	1–5	5	15	3.0
1	6–9	4	30	7.5
2	10–13	4	46	11.5
3	14–17	4	62	15.5
Combined	1–17	17	153	$153/17 = 9.0$

The unweighted mean of the four local averages is 9.375, demonstrating why local averages should not simply be averaged.

4.3.2 How the Balanced Chunks Are Built

The Python expression `divmod(n, size)` returns the quotient q and remainder. Each of the first `remainder` ranks gets one extra element. In Python, the comparison `r < remainder` evaluates to `True` or `False`; in the count expression these act as 1 or 0. The start expression accounts for the extra elements already assigned to lower ranks.

Rank r	$r < 1$	Count	Start	Slice	Chunk from $1, \ldots, 17$
0	True	5	0	`data[0:5]`	$1, 2, 3, 4, 5$
1	False	4	5	`data[5:9]`	$6, 7, 8, 9$
2	False	4	9	`data[9:13]`	$10, 11, 12, 13$
3	False	4	13	`data[13:17]`	$14, 15, 16, 17$

For $N = 17, P = 4$, `zip(starts, counts)` pairs $(0, 5)$, $(5, 4)$, $(9, 4)$, and $(13, 4)$. Each pair (s, c) produces the half-open slice `data[s:s+c]`. These slices cover the input exactly once, with no gaps or overlaps. If $N = 3, P = 4$, counts are $(1, 1, 1, 0)$ and starts are $(0, 1, 2, 3)$; rank 3 receives `data[3:3]`, an empty list. It still calls the subsequent reductions with local sum and count both zero.

The average program validates command-line input on rank 0, then broadcasts either N or an invalid-input marker. All ranks can therefore exit consistently before entering the scatter. Letting only rank 0 exit before a collective would leave other ranks waiting.

4.4 Uneven Numerical Data: Scatterv and Dot Product

An ordinary fixed-count `Scatter` requires the same receive count at every rank. `Scatterv` allows unequal counts. Let $N = 17$ and $P = 4$. Balanced blocks have counts $(5, 4, 4, 4)$ and starting displacements $(0, 5, 9, 13)$. A displacement is measured in *elements* of the MPI datatype, not bytes.

Rank	Count	Displacement	Half-open range
0	5	0	$[0, 5)$
1	4	5	$[5, 9)$
2	4	9	$[9, 13)$
3	4	13	$[13, 17)$

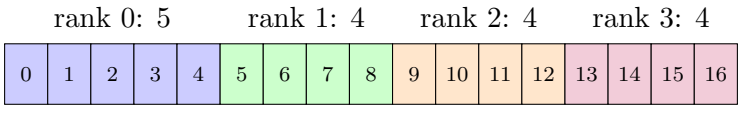


Figure 4.2: Seventeen contiguous array indices split among four ranks. Color boundaries match the counts and displacements in the table.

For vectors x and y, rank r computes

$$d_r = \sum_{i \in B_r} x_i y_i, \qquad x \cdot y = \sum_{r=0}^{P-1} d_r.$$

The complete example sends two `float64` arrays with `Scatterv`, computes local dot products with NumPy, and reduces the partial results. The root uses $x = (1, \ldots, N)$ and $y = (2, \ldots, 2)$, so for $N = 17$ the expected dot product is $2(1 + \cdots + 17) = 306$.

Listing 4.3: `dot_product_scatterv.py`: variable-sized numerical blocks

```python
import sys
from mpi4py import MPI
import numpy as np


comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

n = None
if rank == 0:
    try:
        if len(sys.argv) > 2:
            raise ValueError
        n = int(sys.argv[1]) if len(sys.argv) == 2 else 17
        if n < 0:
            raise ValueError
    except ValueError:
        n = None
n = comm.bcast(n, root=0)
if n is None:
    if rank == 0:
        print("Usage: dot_product_scatterv.py [N], where N >= 0")
    raise SystemExit(1)

q, remainder = divmod(n, size)
counts = np.array([q + (r < remainder) for r in range(size)],
                  dtype=np.int32)
displacements = np.zeros(size, dtype=np.int32)
displacements[1:] = np.cumsum(counts[:-1])

local_n = int(counts[rank])
local_x = np.empty(local_n, dtype=np.float64)
local_y = np.empty(local_n, dtype=np.float64)

if rank == 0:
    x = np.arange(1, n + 1, dtype=np.float64)
    y = np.full(n, 2.0, dtype=np.float64)
else:
    x = y = None

comm.Scatterv([x, counts, displacements, MPI.DOUBLE],
              local_x, root=0)
comm.Scatterv([y, counts, displacements, MPI.DOUBLE],
              local_y, root=0)

local_dot = float(np.dot(local_x, local_y))
global_dot = comm.reduce(local_dot, op=MPI.SUM, root=0)

if rank == 0:
    expected = float(np.dot(x, y))
    print(f"N={n} P={size} dot={global_dot:.1f} "
          f"expected={expected:.1f} "
          f"{'PASS' if np.isclose(global_dot, expected) else 'FAIL'}")
```

```
mpiexec -n 4 python -m mpi4py dot_product_scatterv.py 17
```

For $N < P$, some receive arrays have length zero. All ranks still call both `Scatterv` operations and the reduction. `MPI.DOUBLE` matches `np.float64` in this example. A floating-point dot product may differ slightly from a serial result because additions can occur in a different order; `np.isclose` is used for validation.

Read the `Scatterv` call as a data-layout contract: `counts[r]` gives rank r's receive length, `displacements[r]` gives its starting position in the root array, and `local_x` has exactly `counts[rank]` elements. The two vectors use the same layout. Printing each rank's half-open index range is a useful first check if the dot product is wrong.

4.5 Nonblocking Communication and Request Lifetimes

`Isend` and `Irecv` start buffer operations and return request objects. The program must complete them with `Wait`, `Waitall`, or a suitable test operation. Posting communication early can permit useful independent computation while messages move, but an MPI implementation and workload do not guarantee overlap or speedup.

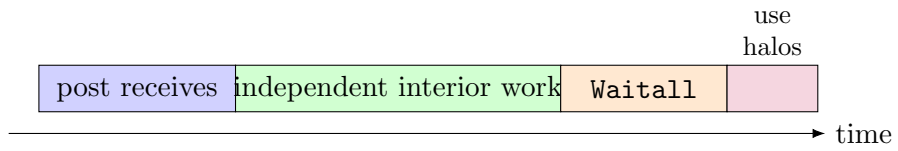


Figure 4.3: A legal sequence: post communication, do work that does not touch active buffers, wait, then use received halo values. Durations are conceptual, not measured.

Buffer state	Rule
Active nonblocking send	Keep the NumPy array alive and do not modify the sent region before completion.
Active nonblocking receive	Do not read or write the receive region before completion.
After `Waitall`	Sent storage can be reused and received values can be read.

Keep the returned request objects until completion. Code that starts an operation and immediately waits is valid, but may leave no independent work to overlap. Never assume that a request has completed merely because the Python call to `Isend` or `Irecv` returned.

4.6 Domain Decomposition and Halo Exchange

In a stencil computation, each rank owns a consecutive part of a domain. A rank also stores *ghost cells* that hold its neighbors' boundary values. For a one-dimensional line with four owned cells per rank, indices 1–4 are owned, index 0 is the left ghost, and index 5 is the right ghost. A boundary rank has no neighbor on one side; `MPI.PROC_NULL` makes that send or receive a safe no-op.

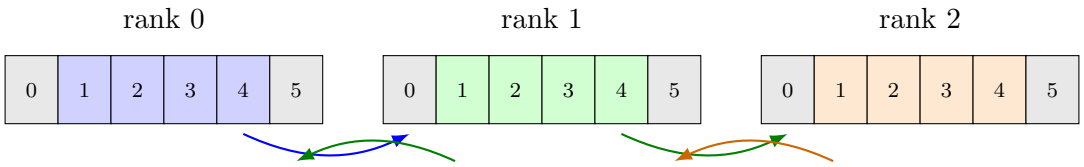


Figure 4.4: Each rank stores four owned cells and two gray ghost cells. Boundary values travel into the neighboring ghost cells; the outermost ghosts have no neighbor.

The tags in a two-sided exchange must match the direction of the data. Here tag 20 identifies a value moving to the right and tag 21 a value moving to the left. A receive from the left therefore expects tag 20, while a receive from the right expects tag 21.

Listing 4.4: `nonblocking_halo.py`: exchange one value with each neighbor

```
from mpi4py import MPI
import numpy as np


comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

left = rank - 1 if rank > 0 else MPI.PROC_NULL
right = rank + 1 if rank < size - 1 else MPI.PROC_NULL

# Indices 1..4 are owned; indices 0 and 5 are ghost cells.
field = np.full(6, -1.0, dtype=np.float64)
field[1:5] = rank

requests = [
    comm.Irecv(field[0:1], source=left, tag=20),
    comm.Irecv(field[5:6], source=right, tag=21),
    comm.Isend(field[1:2], dest=left, tag=21),
    comm.Isend(field[4:5], dest=right, tag=20),
]

# This work does not need ghost cells.
interior_total = float(np.sum(field[2:4]))

MPI.Request.Waitall(requests)

for r in range(size):
    comm.Barrier()
    if rank == r:
        print(f"rank={rank} left={left} right={right} "
              f"field={field.tolist()} interior={interior_total}",
              flush=True)
```

```
mpiexec -n 4 python -m mpi4py nonblocking_halo.py
```

With four ranks, rank 1 should print a field like `[0.0, 1.0, 1.0, 1.0, 1.0, 2.0]`. Rank 0 retains `-1.0` in its outer left ghost, and rank 3 retains it in its outer right ghost. These sentinel values are for illustration; a real simulation would apply explicit physical boundary conditions. The barriers in the final printing loop only order the display. They are not part of the halo exchange itself.

4.6.1 From a Halo to One Stencil Update

A stencil uses the values in adjacent cells. For a one-dimensional heat step with a fixed-zero value beyond each physical end, one possible update is

$$u_i^{\text{new}} = u_i^{\text{old}} + \lambda\big(u_{i-1}^{\text{old}} - 2u_i^{\text{old}} + u_{i+1}^{\text{old}}\big), \qquad 0 < \lambda \le \tfrac{1}{2}.$$

The old and new arrays must be separate so every cell reads values from the same time step. For $N = 5$ and $\lambda = 0.25$, a single impulse in the middle produces:

Global index	0	1	2	3	4
u^{old}	0	0	1	0	0
u^{new}	0	0.25	0.50	0.25	0

For a distributed block, post halo receives and sends first. Update only cells whose two neighbors are already owned locally. After `Waitall`, update the cells at the local edges using the received ghosts. A block of one owned cell has no independent interior and must be updated only once after both halo operations complete. A physical-boundary ghost must be reset to zero on every step; a `PROC_NULL` receive leaves its buffer unchanged. An empty rank posts no halo messages but still enters any global collective in the same order as active ranks.

The accompanying assignment asks students to turn this one-step idea into an iterative program with convergence. The chapter deliberately leaves that full implementation for the assignment.

4.7 Communication Cost and Message Size

A simple model for an n-byte message is

$$T_{\text{message}}(n) = \alpha + \beta n,$$

where α is startup latency and β is transfer time per byte. This model omits contention, serialization, memory copies, and other effects. It is useful for reasoning about message aggregation, not predicting exact hardware timings.

Suppose $\alpha = 5\,\mu$s and $\beta = 1\,$ns/byte. One 8,000-byte message costs $5 + 8 = 13\,\mu$s in this model. Eight 1,000-byte messages cost $8(5 + 1) = 48\,\mu$s. The payload is the same, but the repeated startup cost differs.

Messages	Bytes per message	Total bytes	Model time (μs)
1	8,000	8,000	13
2	4,000	8,000	18
4	2,000	8,000	28
8	1,000	8,000	48

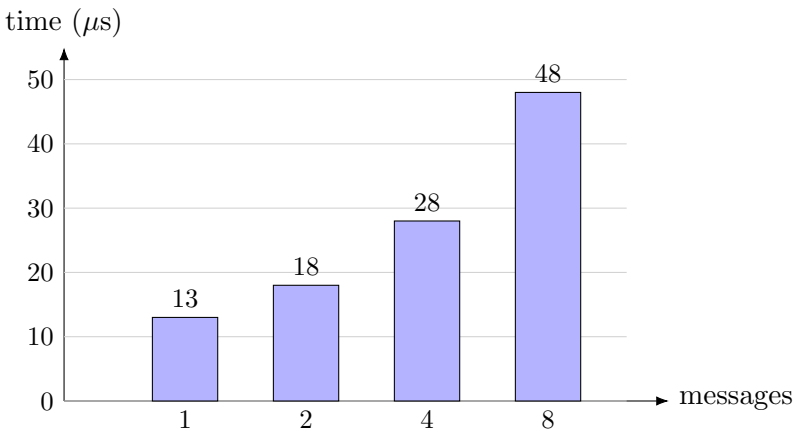


Figure 4.5: Analytical comparison for a fixed 8,000-byte payload using the stated latency-bandwidth model. These are calculated values, not measurements.

Aggregation can reduce overhead, but delaying a message can postpone dependent work. Measure the actual application before choosing a batching strategy.

4.7.1 Measuring a Parallel Phase

Use `MPI.Wtime()` to measure elapsed time around the same fixed-work phase on every rank. A barrier immediately before the timed region can align the start; avoid barriers inside each short iteration unless the algorithm requires them. Reduce the elapsed times with `MPI.MAX`: the slowest rank determines when the phase is finished.

```
comm.Barrier()
t0 = MPI.Wtime()
# Run the same fixed number of computation steps on every rank.
local_elapsed = MPI.Wtime() - t0
phase_time = comm.reduce(local_elapsed, op=MPI.MAX, root=0)
```

Run each configuration several times and report the median of the maximum rank times. Keep the problem size and step count fixed when comparing $P = 1, 2, 4$ ranks. If $T(P)$ is the measured time with P ranks, speedup is $S(P) = T(1)/T(P)$ and efficiency is $E(P) = S(P)/P$. The blank table is a reporting template, not a set of measured results.

Ranks P	Median $T(P)$	Speedup $S(P)$	Efficiency $E(P)$
1	_____	1.00	1.00
2	_____	_____	_____
4	_____	_____	_____

Label the machine, MPI version, and placement of ranks. Do not infer speedup from the analytical message-cost graph above; that graph is calculated from assumed α and β, whereas $T(P)$ must be measured.

4.8 Synchronization, Deadlock, and Load Balance

A barrier makes all ranks wait for the last arrival. Placing it inside every short iteration can expose variation in rank progress and add unnecessary waiting. Some algorithms require a global phase boundary; others need only neighbor exchange or a reduction. Collective call order must remain consistent even when a branch takes different work on each rank.

The time for a parallel phase is at least the time taken by its slowest rank. If four ranks each receive four tasks, but task costs vary, equal *counts* do not ensure equal *times*. The following example is illustrative:

Rank	0	1	2	3
Tasks	4	4	4	4
Work units	9	4	7	4

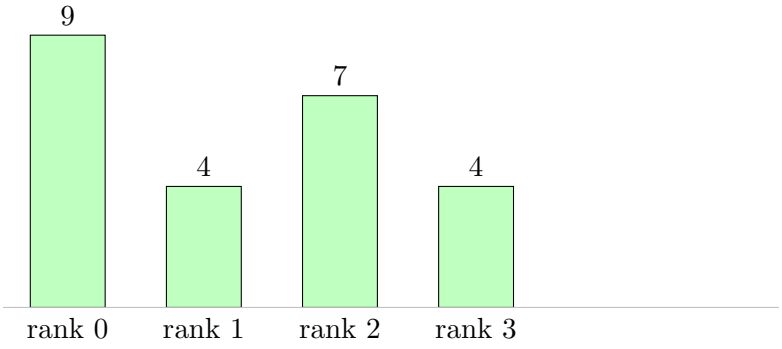


Figure 4.6: Calculated work units for four ranks with equal task counts. The phase waits for rank 0's nine units.

Cyclic assignment can spread some irregular costs. For unpredictable tasks, smaller chunks and a worker queue can improve balance, although scheduling and communication add overhead. Do not claim speedup from a work-count graph; timing requires repeated measurements on specified hardware.

4.9 Debugging and Validation Checklist

Symptom	Likely cause	First check
Collective hangs	Different call order or missing participant	Trace each rank's collective sequence.
Incorrect average	Averaged rank averages	Reduce sums and counts separately.
Wrong dot product	Bad counts, displacements, or dtype	Print owned index ranges and local products.
Stale halo	Wrong tag/neighbor or missing wait	Trace direction tags and request completion.
Inconsistent timing	Placement, warm-up, or imbalance	Record environment and repeat runs.

Start with one rank, then test uneven and empty partitions. Check the serial result and, when possible, an independent mathematical result. Compare floating-point values with a tolerance appropriate to the scale and algorithm rather than assuming bitwise equality. Prefix debug output with rank and flush it; line order alone does not establish computation order.

4.10 Guided Laboratory

1. Run the average program for $N = 17$ with one, two, and four ranks. Record the sum, count, and average. Explain why averaging local averages is unsafe for uneven blocks.
2. Run the average program for $N = 3, P = 4$. Identify the empty rank and show its contributions to the two reductions.
3. Run `collective_results.py` for $N = 7, P = 4$ and $N = 3, P = 4$. Record what rank 0 receives from `gather`, `Gatherv`, and `reduce`, and what every rank receives from `allreduce`.
4. Calculate counts and displacements for $N = 23, P = 5$ by hand. Then modify the dot-product example to use $N = 23$ and inspect each rank's local array length.
5. Run the dot product for $N = 0$ and $N = 3, P = 4$. Explain why all ranks still enter the collectives.
6. Run the halo program with two and four ranks. Draw the messages and write down expected ghost values for every rank.
7. Change the halo example so each rank owns eight values. Keep the ghost locations and tag directions correct.
8. Calculate one heat-stencil update by hand for $[0, 0, 1, 0, 0]$ with $\lambda = 0.25$. Mark which cells would need a halo if the rod were split over two ranks.
9. Use the cost model for a 16,000-byte payload split into 1, 2, 4, or 8 messages. Plot the analytical times and state the model's limitations.
10. Time one fixed-work case with one, two, and four ranks. Report the median of at least three runs, then calculate speedup and efficiency. Label these as measured results.

Submit source code, command lines, a completed result table, one message diagram, and a brief explanation of one bug or edge case. Label analytical graphs and measured timing plots distinctly.

4.11 Summary

- Collectives encode common one-to-many, many-to-one, and all-rank communication patterns.
- `gather` and `Gatherv` collect data at a root; `reduce` combines values at a root; `allreduce` combines values for every rank.
- Uneven data can be distributed as Python-object chunks or with buffer counts and displacements.
- A global average requires global sum divided by global count.
- Nonblocking requests require explicit completion and careful buffer ownership.
- Halo exchange moves boundary values into ghost cells; direction, neighbor, and tag must agree.
- Latency, bandwidth, synchronization, and imbalance jointly influence runtime.

4.12 Exercises

1. State which ranks receive a result from `reduce` and from `allreduce`. Give one use case for each.
2. Why must ranks call compatible collectives in the same order?
3. For $N = 19, P = 4$, calculate balanced counts, displacements, and half-open ranges.
4. Explain why the mean of $(3.0, 7.5, 11.5, 15.5)$ is not the global average in the worked example.
5. Modify the average program to calculate a global maximum. Define how an empty rank contributes.
6. For $x = (1, 2, 3, 4)$ and $y = (2, 2, 2, 2)$, calculate each local dot product for two balanced ranks and the reduced answer.
7. If $N = 2, P = 4$, which ranks receive zero elements in `Scatterv`? What must they still call?
8. Identify which arrays in `nonblocking_halo.py` must remain unchanged until `Waitall`.
9. Draw both halo messages between ranks 1 and 2 and annotate their tags.
10. Explain the role of `MPI.PROC_NULL` at a physical boundary.
11. Using $\alpha = 5\,\mu$s and $\beta = 1$ ns/byte, compare one 16,000-byte message with four 4,000-byte messages.
12. Why can adding a barrier after every short computation step reduce scalability?
13. Give a test that would expose a wrong `Scatterv` displacement and another that would expose a reversed halo tag.
14. Explain why a correct analytical work-count graph cannot by itself establish runtime speedup.
15. For $N = 3, P = 4$, list each rank's count, start, and chunk produced by `zip(starts, counts)`.
16. Explain why a `PROC_NULL` receive does not establish a fixed-zero physical boundary on its own in an iterative stencil.
17. If measured times are $T(1) = 12$ s, $T(2) = 7$ s, and $T(4) = 4$ s, calculate speedup and efficiency at two and four ranks.

4.13 Selected Worked Answers

Exercise 3. For $N = 19, P = 4$, $q = 4$ and the remainder is 3. Counts are $(5, 5, 5, 4)$; displacements are $(0, 5, 10, 15)$; ranges are $[0, 5)$, $[5, 10)$, $[10, 15)$, and $[15, 19)$.

Exercise 4. The unweighted mean of the local averages is 9.375. The global average is $(15 + 30 + 46 + 62)/(5 + 4 + 4 + 4) = 153/17 = 9$. Local averages have unequal weights.

Exercise 6. Rank 0 owns $(1, 2)$ and computes $2 + 4 = 6$. Rank 1 owns $(3, 4)$ and computes $6 + 8 = 14$. The reduced dot product is 20.

Exercise 11. One message costs $5 + 16 = 21\,\mu$s. Four messages cost $4(5 + 4) = 36\,\mu$s under the model. This excludes overlap and other system effects.

Exercise 15. Counts are $(1, 1, 1, 0)$, starts are $(0, 1, 2, 3)$, and the chunks are [1], [2], [3], and []. The fourth rank still participates in the collectives.

Exercise 17. At two ranks, speedup is $12/7 \approx 1.71$ and efficiency is about 0.86. At four ranks, speedup is $12/4 = 3$ and efficiency is 0.75. These times are illustrative exercise inputs, not measured results from this chapter.

4.14 Further Reading

The MPI for Python documentation describes the object and buffer APIs, vector collectives, requests, and their argument conventions:

- `https://mpi4py.readthedocs.io/en/stable/tutorial.html`
- `https://mpi4py.readthedocs.io/en/stable/overview.html`
- `https://mpi4py.readthedocs.io/en/stable/reference/mpi4py.MPI.Comm.html`
- `https://mpi4py.readthedocs.io/en/stable/reference/mpi4py.MPI.Request.html`
- `https://docs.open-mpi.org/`